

Django Logging System

Django's logging system is built on top of Python's built-in logging module.

Logging is useful for:

- Debugging application problems
- Tracking errors
- Monitoring application behavior
- Recording warnings
- Troubleshooting production issues
- Keeping application events in persistent files

1. What is Logging?

Logging means recording information about what is happening inside an application.

For example:

```
javascript.snippet JAVASCRIPT
User logged inOrder createdPayment failedDatabase connection failedInvalid request
```

Instead of using:

```
javascript.snippet JAVASCRIPT
print("Something happened")
```

we can use:

```
javascript.snippet JAVASCRIPT
logger.info("Something happened")
```

Logging is much more suitable for real applications because it provides:

- Log levels
- Timestamps
- Formatting
- File output
- Console output

- Error tracking
- Log rotation

2. Django Logging Levels

Python logging provides several levels:

```
javascript.snippet JAVASCRIPT  
  
DEBUGINFOWARNINGERRORCRITICAL
```

The most commonly used levels are:

Level	Meaning
DEBUG	Detailed information for development
INFO	Normal application events
WARNING	Something unexpected but not necessarily an error
ERROR	An operation failed
CRITICAL	A serious failure that may affect the application

Example:

```
javascript.snippet JAVASCRIPT  
  
logger.debug("Debug information")logger.info("User logged in")logger.warning("Low stock  
")logger.error("  
Payment failed")logger.critical("Database is unavailable")
```

3. Basic Logging Setup

In settings.py:

```
javascript.snippet JAVASCRIPT

import os from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "verbose": {
            "format": "{levelname} {asctime} {module} {message}",
            "style": "%(asctime)s %(levelname)s %(module)s %(message)s",
        },
        "simple": {
            "format": "{levelname} {message}",
            "style": "%(levelname)s %(message)s",
        },
    },
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
            "level": "INFO",
            "formatter": "verbose",
        },
        "file": {
            "class": "logging.FileHandler",
            "level": "INFO",
            "formatter": "verbose",
            "filename": BASE_DIR / "logs/app.log",
        },
    },
    "loggers": {
        "django": {
            "handlers": ["console", "file"],
            "level": "INFO",
            "propagate": False,
        },
    },
}
```

4. Create the logs Directory

The directory must exist before using `FileHandler`.

From the project root:

```
javascript.snippet JAVASCRIPT

mkdir logs
```

Project structure:

```
javascript.snippet JAVASCRIPT

myproject/ manage.py logs/ app.log myproject/ settings.py urls.py wsgi.py
app
/
```

If the directory does not exist, Django may fail to start because `FileHandler` cannot create the missing directory.

5. Getting a Logger in Django

In `views.py`:

```
javascript.snippet
```

JAVASCRIPT

```
import logging  
logger = logging.getLogger(__name__)
```

Then:

```
javascript.snippet
```

JAVASCRIPT

```
def my_view(request):  
    logger.info("my_view was called")  
    logger.warning("This is a warning")  
    logger.error("Something went wrong")  
    return HttpResponse("Check the logs")
```

Using:

```
javascript.snippet
```

JAVASCRIPT

```
logging.getLogger(__name__)
```

is generally preferred over:

```
javascript.snippet
```

JAVASCRIPT

```
logging.getLogger("django")
```

because it gives each module its own logger name.

For example:

```
javascript.snippet
```

JAVASCRIPT

```
myapp.views  
myapp.models  
myapp.services  
myapp.tasks
```

This makes logging easier to organize.

6. Separate INFO, WARNING and ERROR Logs

If you specifically want:

```
javascript.snippet JAVASCRIPT

logs/ info.log warning.log error.log
```

then simply assigning:

```
javascript.snippet JAVASCRIPT

"level": "INFO"
```

to `info.log` is **not enough**.

Why?

Logging levels are thresholds.

Therefore:

```
javascript.snippet JAVASCRIPT

INFO handler    INFO WARNING ERROR CRITICAL
```

The handler accepts everything at or above its level.

Similarly:

```
javascript.snippet JAVASCRIPT

WARNING handler  WARNING ERROR CRITICAL
```

And:

```
javascript.snippet JAVASCRIPT

ERROR handler    ERROR CRITICAL
```

7. Exact-Level Logging with Filters

If you want strictly separate files, create filters.

```

import loggingfrom pathlib import PathBASE_DIR = Path(__file__).resolve().parent.
parentclass
    InfoFilter(logging.Filter):    def filter(self, record):        return record.levelno
==
    logging.INFOclass WarningFilter(logging.Filter):    def filter(self, record):
return
    record.levelno == logging.WARNINGclass ErrorFilter(logging.Filter):    def filter(self
,
    record):        return record.levelno == logging.ERROR

```

Then configure the handlers:

```

LOGGING = {    "version": 1,    "disable_existing_loggers": False,    "formatters": {
"verbose": {        "format": "{levelname} {asctime} {name} {module} {message}",
"style": "{",        },        },    "filters": {        "only_info": {            "()":
InfoFilter
,            },        "only_warning": {            "()": WarningFilter,        },
"only_error"
: {            "()": ErrorFilter,        },        },    "handlers": {        "file_info":
{
            "class": "logging.FileHandler",            "filename": BASE_DIR /
"logs/info.log"
,            "level": "INFO",            "filters": ["only_info"],            "
formatter
": "verbose",        },        "file_warning": {            "class": "logging.
FileHandler
",            "filename": BASE_DIR / "logs/warning.log",            "level": "WARNING
",            "
filters": ["only_warning"],            "formatter": "verbose",        },        "
file_error
": {            "class": "logging.FileHandler",            "filename": BASE_DIR / "logs
/
error.log",            "level": "ERROR",            "filters": ["only_error
"],            "
formatter": "verbose",        },        },    "loggers": {        "myapp": {            "
handlers
": [                "file_info",                "file_warning",                "
file_error
",            ],            "level": "INFO",            "propagate": False,        },
},
},

```

Here myapp should be replaced with your actual Django application name.

8. Using the Logger

Inside your application:

```
javascript.snippet
```

JAVASCRIPT

```
import logging
logger = logging.getLogger(__name__)
```

Example:

```
javascript.snippet
```

JAVASCRIPT

```
def my_view(request):
    logger.info("User visited the page")
    logger.warning("User submitted
an unusual request")
    logger.error("Payment processing failed")
    return HttpResponse("Done")
```

Now:

```
javascript.snippet
```

JAVASCRIPT

```
logs/ info.log warning.log error.log
```

will contain their corresponding exact levels.

9. Logging Exceptions

One of the most useful features is `logger.exception()`.

Example:

```
javascript.snippet
```

JAVASCRIPT

```
import logging
logger = logging.getLogger(__name__)

def calculate():
    try:
        result = 10 / 0
        return result
    except Exception:
        logger.exception("Calculation failed")
        return None
```

The log will contain the error and traceback.

This is much better than:

```
logger.error("Calculation failed")
```

because `logger.exception()` automatically includes the traceback when called inside an exception handler.

10. `logger.error()` vs `logger.exception()`

`logger.error()`

```
logger.error("Payment failed")
```

Records an error message.

`logger.exception()`

```
try: ...except Exception: logger.exception("Payment failed")
```

Records:

```
ERROR+Exception message+Traceback
```

For debugging exceptions, `logger.exception()` is usually more useful.

11. Logging Variables

Avoid constructing strings manually when possible.

Instead of:

```
javascript.snippet JAVASCRIPT

logger.info("User %s logged in" % user.username)
```

USE:

```
javascript.snippet JAVASCRIPT

logger.info(    "User %s logged in",    user.username)
```

Another example:

```
javascript.snippet JAVASCRIPT

logger.info(    "Order %s created by user %s",    order.id,    request.user.id,)
```

This follows Python logging's built-in formatting mechanism.

12. Logging in Models

You can also log from models.

```
javascript.snippet JAVASCRIPT

import logging
logger = logging.getLogger(__name__)

class Product(models.Model):
    name = models.CharField(max_length=100)
    stock = models.IntegerField(default=0)

    def reduce_stock(self, quantity):
        logger.info(
            "Reducing stock for product %s by %s",
            self.id, quantity,
        )
        self.stock -= quantity
        self.save()
```

13. Logging in Services

For larger Django projects, business logic is often placed in service modules.

Example:

```
javascript.snippet JAVASCRIPT

app/ views.py models.py services.py tasks.py
```

In services.py:

```
javascript.snippet JAVASCRIPT

import logging
logger = logging.getLogger(__name__)

def create_order(user, product):
    logger
    .info(
        "Creating order for user=%s product=%s",
        user.id,
        product.
    id
    ,
    ) # Business logic
```

This makes debugging complex applications easier.

14. Logging with DEBUG

You can use:

```
javascript.snippet JAVASCRIPT

logger.debug("Variable value: %s", value)
```

For development:

```
javascript.snippet JAVASCRIPT

DEBUG INFO WARNING ERROR CRITICAL
```

But production logging should generally avoid excessive debug output.

15. Console Logging

During development, console logging is very useful.

```
javascript.snippet JAVASCRIPT

"console": {
    "class": "logging.StreamHandler",
    "level": "DEBUG",
    "formatter":
    "verbose"
, }
```

Then logs appear directly in:

```
javascript.snippet JAVASCRIPT

Terminal
```

Example:

```
javascript.snippet JAVASCRIPT
INFO 2026-10-03 12:30:22 views User logged inWARNING 2026-10-03 12:30:30 views Low
stockERROR
2026-10-03 12:31:02 views Payment failed
```

16. File Logging vs Console Logging

Console

```
javascript.snippet JAVASCRIPT
Application Terminal
```

Good for:

```
javascript.snippet JAVASCRIPT
DevelopmentDockerKubernetesCloud environments
```

File

```
javascript.snippet JAVASCRIPT
Application Log file
```

Good for:

```
javascript.snippet JAVASCRIPT
Local debuggingSimple server deploymentsPersistent local logs
```

In production, centralized logging systems are often preferred over keeping large log files on the application server.

17. Log Rotation

Using:

```
javascript.snippet JAVASCRIPT

logging.FileHandler
```

means the file can continue growing indefinitely.

For production applications, a rotating handler is often better.

Example:

```
javascript.snippet JAVASCRIPT

"file": {    "class": "logging.handlers.RotatingFileHandler",    "filename": BASE_DIR /
"logs/app.log",    "maxBytes": 10 * 1024 * 1024,    "backupCount": 5,    "level":
"INFO"
,    "formatter": "verbose",}
```

Here:

```
javascript.snippet JAVASCRIPT

maxBytes = 10 MBbackupCount = 5
```

When the current log becomes too large, older logs are rotated.

18. Timed Log Rotation

You can also rotate logs based on time.

```
javascript.snippet JAVASCRIPT

"file": {    "class": "logging.handlers.TimedRotatingFileHandler",    "filename":
BASE_DIR
/ "logs/app.log",    "when": "midnight",    "backupCount": 7,    "level": "INFO",
"formatter"
: "verbose",}
```

This creates a new log file periodically.

For example:

```
javascript.snippet JAVASCRIPT
app.logapp.log.2026-10-.log.2026-10-.log.2026-10-03
```

19. Recommended Project Structure

A simple project can look like:

```
javascript.snippet JAVASCRIPT
myproject/ manage.py logs/ info.log warning.log error.log config/ settings.
py
    urls.py asgi.py wsgi.py shop/ models.py views.py services.py
urls
.py
```

20. Important Production Considerations

1. Do not log sensitive information

Never log:

```
javascript.snippet JAVASCRIPT
PasswordsAccess tokensAPI keysCredit card informationSession secrets
```

Bad:

```
javascript.snippet JAVASCRIPT
logger.info("Password: %s", password)
```

Never do this.

2. Use meaningful messages

Bad:

```
javascript.snippet JAVASCRIPT

logger.error("Error")
```

Better:

```
javascript.snippet JAVASCRIPT

logger.error("Payment failed for order=%s", order.id,)
```

Even better when handling an exception:

```
javascript.snippet JAVASCRIPT

logger.exception("Payment processing failed for order=%s", order.id,)
```

21. propagate

Example:

```
javascript.snippet JAVASCRIPT

"myapp": { "handlers": ["file"], "level": "INFO", "propagate": False,}
```

`propagate=False` prevents the log record from being passed to parent loggers.

This can help prevent duplicate log entries when you have your own application handlers.

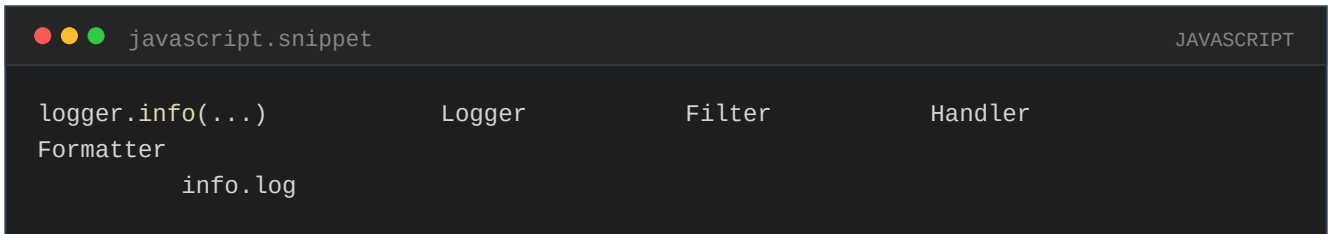
22. Logger vs Handler vs Formatter vs Filter

This is very important.

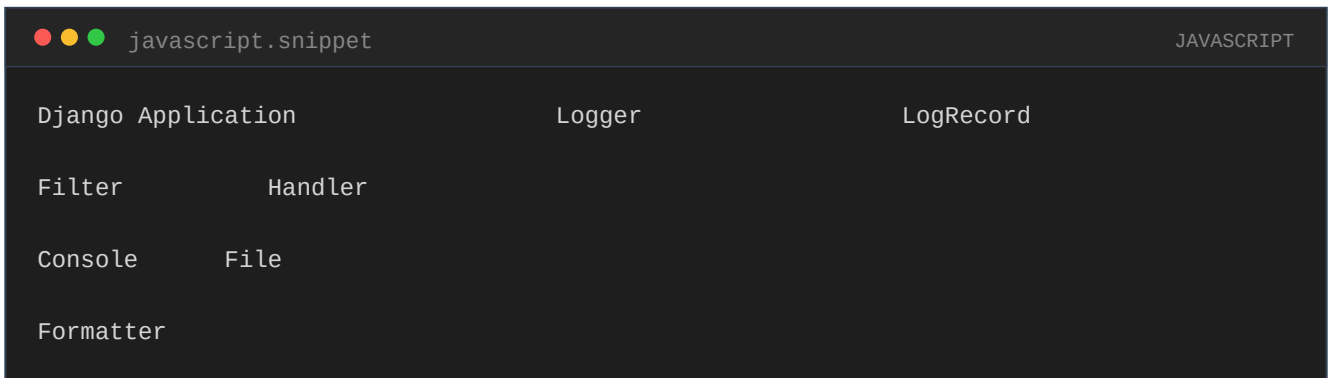
```
javascript.snippet JAVASCRIPT

Logger    Creates/receives log records
Handler   Decides where logs go
Formatter Decides how logs look
Filter    Decides which records are allowed
```

Example:



23. Complete Mental Model



24. Quick Example

views.py:

```

    javascript.snippet
    JAVASCRIPT

    import logging
    from django.http import HttpResponse
    logger = logging.getLogger(__name__)

    def home(request):
        logger.info("Home page accessed")
        try:
            result = 10 / 0
        except ZeroDivisionError:
            logger.exception("Calculation failed")
        return HttpResponse("Home Page")
  
```

Possible output:

```

    javascript.snippet
    JAVASCRIPT

    INFO 2026-10-03 20:00:00 views Home page accessed
    ERROR 2026-10-03 20:00:00 views Calculation failed
    Traceback ...
  
```

25. Final Summary

Logging

```
javascript.snippet JAVASCRIPT

Logging Record application events
```

Log Levels

```
javascript.snippet JAVASCRIPT

DEBUGINFOWARNINGERRORCRITICAL
```

Main Components

```
javascript.snippet JAVASCRIPT

LoggerHandlerFormatterFilter
```

Common Handlers

```
javascript.snippet JAVASCRIPT

StreamHandlerFileHandlerRotatingFileHandlerTimedRotatingFileHandler
```

Django Usage

```
javascript.snippet JAVASCRIPT

import logginglogger = logging.getLogger(__name__)
```

Then:

```
javascript.snippet JAVASCRIPT

logger.debug(...)logger.info(...)logger.warning(...)logger.error(...)logger.exception(...)
logger.critical(...)
```

Recommended Production Pattern

```
javascript.snippet JAVASCRIPT

Django Logger Handlers Console Rotating File Logs
```

For a small project:

```
logs/ info.log warning.log error.log
```

For a production project, consider **rotating logs** and/or sending logs to a centralized logging system instead of allowing local files to grow indefinitely.